Enhancing Drupal with PHP8: Exploring New Features and Compatibility

Presented by: Ajit Shinde

https://bit.ly/ajits-php8-singapore

Ajit Shinde

- Sr. Drupal Developer at Tag1
- Mostly work with Backend - PHP :)
- Trainer and mentor
- One of the core organizers of Drupal Pune Group
- https://drupal.org/u/ajits

- #2 all-time contributor to Drupal
- Global team of 100+ experts
- Largest concentration of Core contributors at any organization
- We contribute a full time senior infra expert to Drupal Association

We are *(almost)* always hiring... ;)

tag1

Montefiore Einstein

ALBEMARLE

THE WHITE HOUSE WASHINGTON

Top 50 CONFIDENTIAL Global 500

Google

Sumitomo

itv

NTT

RENESAS

ACLU

tag1

Symantec

THE LINUX FOUNDATION

ALJAZEERA

AMERICAN FEDERATION OF LABOR & CONGRESS OF INDUSTRIAL ORGANIZATIONS AFL CIO

Stanford University

Capgemini

The New York Times

MICHIGAN

PANTHEON

EUROPEAN UNION PATENT OFFICE

STATE OF MICHIGAN SECRETARY OF STATE

FORTIVE

Acquia

drop solid

https://bit.ly/ajits-php8-singapore

What can PHP do?

Anything. PHP is mainly focused on
generate dynamic page content, or s

Agenda

- Part I : PHP release cycle; how it affects Drupal
- Part II : PHP8 and new features

PART I:

PHP, Drupal, and related project's release cycles

Some history

Year	Milestone	Fun fact
1994	Rasmus Lerdorf creates PHP Tools	Initially stood for *Personal Home Page*. Now it's *PHP: Hypertext Preprocessor*.
1995	PHP/FI released as a scripting language	Over 15,000 installations by 1996, showcasing early popularity.
1997	PHP 3 rewritten by Gutmans & Suraski	Introduced database support, sparking wide adoption.
2000	Zend Engine powers PHP 4	The name *Zend* combines Zeev and Andi's names.
2004	PHP 5 adds OOP and better features	Drupal started partial support since Drupal 6 (2008)
2015	PHP 7 delivers 2x performance boost	Drupal 8 started requiring PHP 7 starting **May 1, 2019**
2020	PHP 8 introduces JIT and modern features	PHP powers almost 80% of all internet (source)

PHP Release & support

Branch	Initial Release		Active Support Until		Security Support Until	
8.1	25 Nov 2021	*3 years ago*	25 Nov 2023	*1 year ago*	31 Dec 2025	*in 1 year*
8.2	8 Dec 2022	*2 years ago*	31 Dec 2024	*in 21 days*	31 Dec 2026	*in 2 years*
8.3	23 Nov 2023	*1 year ago*	31 Dec 2025	*in 1 year*	31 Dec 2027	*in 3 years*
8.4	21 Nov 2024	*18 days ago*	31 Dec 2026	*in 2 years*	31 Dec 2028	*in 4 years*

https://www.php.net/supported-versions.php

PHP Release & support - timeline

https://www.php.net/supported-versions.php

Drupal release milestones

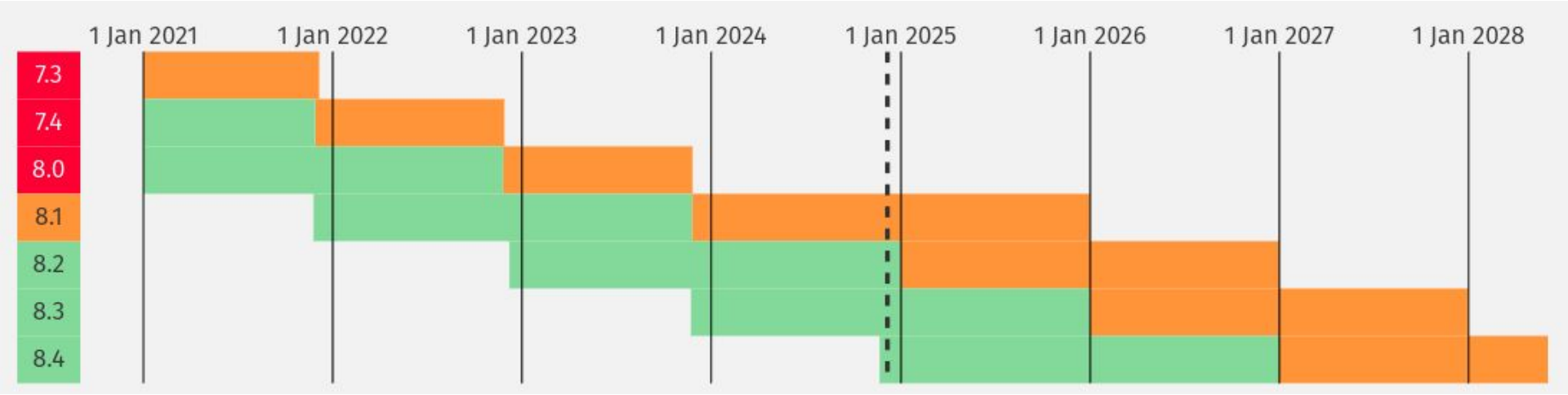
November 19, 2015	Drupal 8.0.0 released.	PHP 5.5.9
February 24, 2016	End-of-life for Drupal 6.	
June 3, 2020	Drupal 9.0.0 and 8.9.0 released.	PHP 7
November 17, 2021	End of life for Drupal 8.	
December 15, 2022	Drupal 10.0.0 and 9.5.0 released.	PHP 8.1 for D10
June 22, 2023	Drupal 10.1.0 released.	PHP 8.1
November 1, 2023	End of life for Drupal 9.	
December 15, 2023	Drupal 10.2.0 released.	PHP 8.3
June 20, 2024	Drupal 10.3.0 released.	PHP 8.3
August 2, 2024	Drupal 11 released.	PHP 8.3

Supported PHP versions

	Drupal version					
PHP version	**10.1**	**10.2**	**10.3**	**10.4**	**11.0**	**11.1**
8.1 (notes)	✅ Yes	✅ Yes	✅ Yes	✅ Yes	❌ No	❌ No
8.2	✅ Yes	✅ Yes	✅ Yes	✅ Yes	❌ No	❌ No
8.3	❌ No	✅ Yes	✅ Yes	✅ Yes	✅ Yes	✅ Yes
8.4	❌ No	❌ No	❌ No	✅ Yes	❌ No	✅ Yes

https://www.drupal.org/docs/getting-started/system-requirements/php-requirements

PHP Release & support - timeline (todo superimpose)



PHP version determination in projects

composer.json

```
"config": {
    "preferred-install": "dist",
    "sort-packages": true,
    "platform": {
        "php": "8.1.0"
    },
```

Simulation

```
},
"require": {
  "php": ">=8.0",
  "drupal/core": "^9.4 || ^10 || ^11",
  "drupal/token": "^1.0"
},
```

Enforcement

PART II:

PHP 8+ : Features

Just In Time Compiler - JIT

Executing code faster than ever

JIT - The Just In Time Compiler

- Extension of OPCache
- Typically PHP code is run in Zend VM
- JIT code runs directly in CPU
- Significantly better performance for arithmetic calculations
- Minute impact on regular PHP applications
- More functions could be written in PHP (C)

Constructor Property Promotion

Streamline class definitions —less boilerplate, more clarity!

Scenario - Book Publishing

Without Constructor Property Promotion

```
class Book {
  1 usage
  private $title;
  1 usage
  private $author;
  1 usage
  private $publicationYear;

  public function __construct($title, $author, $publicationYear) {
    $this->title = $title;
    $this->author = $author;
    $this->publicationYear = $publicationYear;
  }

  // Other class methods...
}

// Example usage:
$book = new Book('Clean Code', 'Robert C. Martin', 2008);
```

With Constructor Property Promotion

```php
class Book {
  public function __construct(
    private $title,
    private $author,
    private $publicationYear
  ) {}
}

// Example usage:
$book = new Book('Clean Code', 'Robert C. Martin', 2008);
```

Constructor Property Promotion in Drupal Core

Drupal\Core\Field\FieldTypeCategoryManager

```
public function __construct(
    protected readonly string $root,
    ModuleHandlerInterface $module_handler,
    CacheBackendInterface $cache_backend
) {
    $this->moduleHandler = $module_handler;
```

Strict Type Hinting:

Enforcing precision at every level

tag1

Without Strict Type Hinting

```php
class Book {
  public function __construct(
    private $title,
    private $author,
    private $publicationYear
  ) {}
}
$book = new Book (123, 23.34, '2024');
```

Syntactically correct ✅

Logically wrong ❌

With Strict Type Hinting

```
class Book {
  1 usage
  private string $title;
  1 usage
  private string $author;
  1 usage
  private int $publicationYear;

  public function __construct(string $title, string $author, int $publicationYear) {
    $this->title = $title;
    $this->author = $author;
    $this->publicationYear = $publicationYear;
  }

  // Other class methods...
}
```

Strict Type Hinting

```php
class Book {
  public function __construct(
    private string $title,
    private string $author,
    private int $publicationYear
  ) {}
}

// Example usage:
$book = new Book('Clean Code', 'Robert C. Martin', 2008);
```

Strict type hinting in Core

Drupal\Core\Ajax\AddJsCommand

```
 */
4 usages   bnjmnm
public function __construct(array $scripts, string $selector = 'body') {
  $this->scripts = $scripts;
  $this->selector = $selector;
}
```

Union types

Multiple types for a single parameter
or return value

Union types

```php
class Book {
  public function __construct(
    private string $title,
    private string $author,
    private int|string $publicationYear // Union type
  ) {}

  // Example getter.
  no usages
  public function getPublicationYear(): int|string {
    return $this->publicationYear;
  }
}

// Example usage:
$book1 = new Book('Clean Code', 'Robert C. Martin', 2008);
$book2 = new Book('Mystery Book', 'Unknown Author', 'Unknown');
```

Union types in Drupal Core

`Drupal\Core\Ajax\OpenModalDialogCommand`

```
/**
 * Constructs an OpenModalDialog object.
 *
 * @param string|\Stringable|null $title
 *   The title of the dialog.
 * @param string|array $content
 *   The content that will be placed in the dialog, either a render
 *   or an HTML string.
 */
5 usages   Alex Pott +1
public function __construct(string|\Stringable|null $title, $content
```

Named arguments

```php
class Book {
  public function __construct(
    private string $title,
    private string $author,
    private int|string $publicationYear // Union type
  ) {}
}
// Example usage with named arguments:
$book1 = new Book(
  title: 'Clean Code',
  author: 'Robert C. Martin',
  publicationYear: 2008
);
$book2 = new Book(
  author: 'Unknown Author',
  title: 'Mystery Book',
  publicationYear: 'Unknown' // Using string as an alternate type
);
```

Attributes

Simplify metadata, adding annotations directly to code

Attributes (replacement of PHPDoc annotations)

```
#[Attribute]
class Required {
  public function __construct(public string $message = "This field is required") {}
}

no usages
#[Attribute]
class MaxLength {
  no usages
  public function __construct(public int $length) {}
}
```

Attributes (implementation)

```php
class Book {
  public function __construct(
    #[Required(message: "Title is mandatory")]
    private string $title,

    #[Required(message: "Author is mandatory")]
    private string $author,

    #[MaxLength(length: 4)]
    private int|string $publicationYear
  ) {}
}
```

In Core - Metadata before attributes

```
/**
 * Provides a 'Help' block.
 *
 * @Block(
 *   id = "help_block",
 *   admin_label = @Translation("Help"),
 *   forms = {
 *     "settings_tray" = FALSE,
 *   },
 * )
 */
no usages
class HelpBlock extends BlockBase implements
```

In Core - With Attributes

```
/**
 * Provides a 'Help' block.
 */
no usages   Nathaniel Catchpole +6
#[Block(
  id: "help_block",
  admin_label: new TranslatableMarkup( string: "Help"),
  forms: ['settings_tray' => FALSE]
)]
class HelpBlock extends BlockBase implements Contain
```

Enums

Defining a set of possible values in a type-safe way

New Status field

```php
class Book {
  public function __construct(
    public string $title,
    public string $author,
    public int|string $publicationYear, // Union type
    public string $status // String type for status
  ) {
    // Validate against only allowed values.
    if (!in_array($status, ['available', 'checked_out', 'reserved'])) {
      throw new InvalidArgumentException("Invalid status value.");
    }
  }
}
```

Introducing Enums

```php
enum BookStatus: string {
  case Available = 'available';
  case CheckedOut = 'checked_out';
  case Reserved = 'reserved';
}
```

```php
class Book {
  public function __construct(
    public string $title,
    public string $author,
    public int|string $publicationYear,
    public BookStatus $status
  ) {}
}
```

```php
$book = new Book(
  title: 'Clean Code',
  author: 'Robert C. Martin',
  publicationYear: 2008,
  status: BookStatus::Available
);
```

Before Enums

`Drupal\Core\File\FileSystemInterface::EXISTS_RENAME`

```
 * @deprecated in drupal:10.3.0 and is removed j
 * \Drupal\Core\File\FileExists::Rename instead.
 *
 * @see https://www.drupal.org/node/3426517
 */
const EXISTS_RENAME = 0;
```

Introducing Enums

```
enum FileExists {

  /* Appends a number until name is unique. */
  case Rename;
  /* Replace the existing file. */
  case Replace;
  /* Do nothing and return FALSE. */
  case Error;
```

Enums - backwards compatibility

```
 *
 * @see https://www.drupal.org/node/3426517
 */
public static function fromLegacyInt(int $legacyInt, string $methodName): self {
  @trigger_error( message: "Passing the \$fileExists argument as an integer to $me
  return match ($legacyInt) {
    0 => FileExists::Rename,
    2 => FileExists::Error,
    default => FileExists::Replace,
  };
}
```

Useful String-related Functions

- str_contains
- str_starts_with
- str_ends_with

Questions?